# VERIQ + CEDAR

## Final Engineering Specification v1.0

**Project:** Veriq — bank-specific contextual intelligence infrastructure
**Simulation institution:** Cedar Bank (fictional Nigerian digital bank)
**Competition priority:** Wema Hackaholics first; preserve adaptability for Zenith Zecathon
**Document type:** Frozen hackathon implementation baseline and team handoff
**Date:** 15 October 2026
**Owners:** Backend — Daniel; AI — Habeeb; Frontend — Usaamah
**Status:** FINAL v1.0 for implementation; changes require a recorded amendment or ADR.

> **Scope statement.** One submitted demo, consisting of two connected acts in the *same Cedar environment*: (1) bank onboarding and construction of a living institutional context; (2) a real interaction in Cedar's simulated customer app, a ₦50,000 transfer with an inconclusive network response, a customer complaint, Veriq's investigation, and later authoritative confirmation. The continuously operating monitoring view bridges both acts. This is a synthetic proof of concept, not a regulated bank, real NIBSS integration, or production-ready payment system.

### 0. Decisions frozen in this document

| ID | Decision | Binding consequence |
|---|---|---|
| D-01 | Veriq is the intelligence product; Cedar is the **independent** fictional bank it serves. | Separate data authority, namespace, and interfaces. |
| D-02 | **One** recorded demo with two linked acts, not two submitted demos. | The knowledge imported in Act I must be used in Act II without resetting it. |
| D-03 | Cedar supports a functional customer-facing **interbank transfer** and complaint journey. | No fake success banner disconnected from server records. |
| D-04 | Main incident: a ₦50,000 transfer to fictional Elm Bank is debited, then response is inconclusive. | Visible financial outcome remains **UNKNOWN** until authorized evidence arrives. |
| D-05 | Main scenario's private simulator truth is **recipient credited, response lost**. | The intelligence engine must not know this until a later status response explicitly exposes it. |
| D-06 | Veriq continuously maintains Cedar's contextual state **before** a complaint is filed. | Telemetry and operational incidents can enter without a customer action. |
| D-07 | Onboarding consumes relevant available bank information, structured and unstructured. | Demo implements selected input types; architecture imposes no artificial 'one narrow kind of knowledge' philosophy. |

| ID | Decision | Binding consequence |
|---|---|---|
| D-08 | AI extracts candidate claims and relationships; backend validates, reconciles and persists. | LLM output cannot overwrite ledger, trusted events or verified financial outcomes. |
| D-09 | One NestJS TypeScript backend, structured as logically separate Cedar and Veriq modules, plus FastAPI AI. | Do not split 12 fictional banking systems into 12 deployed services. |
| D-10 | PostgreSQL + Prisma for app state; deterministic seed and simulator clock for replay. | Hidden ground truth resides in Cedar-only restricted tables, inaccessible from Veriq queries. |
| D-11 | Frontend is one Next.js app with customer and investigator/demo routes. | Role-based server APIs, not merely UI-hidden controls. |
| D-12 | No live payments, real customer information, real bank credentials or real NIBSS access. | All financial balances, bank names and references are fictional. |
| D-13 | Veriq is advisory; investigator authorizes conclusions and customer-safe updates. | No AI-triggered reversal, credit or irreversible financial operation. |
| D-14 | P0 is a working end-to-end vertical slice; queues, embeddings, OCR, graph DB, cloud-scale infra are not P0. | Add technology only when it demonstrably improves the agreed demo. |

## 1. Product purpose and evidence boundary

### 1.1 Product thesis

Veriq maintains an evolving, bank-specific contextual representation of an institution's relevant facts, relationships, operating states, historical knowledge, evidence and uncertainty. It ingests authorized information from heterogeneous sources, uses AI-assisted extraction for unstructured content, reconciles knowledge with deterministic rules, and uses contextual reasoning to support proactive incident detection and targeted investigations. The initial specialization is Nigerian payment operations. Future banking domains may reuse the same foundation through new source mappings and schemas.

### 1.2 What the demo proves

1. Cedar can supply institutional knowledge in structured and unstructured formats.
2. Veriq can extract claims, accept or flag relationships, preserve sources and build a queryable living context.
3. Operational events can update context and surface a suspected degradation **without** a complaint.
4. A customer can initiate a genuine simulated debit and payment instruction, see an unresolved outcome, and report it.
5. Veriq can associate the complaint with eligible transaction evidence and existing contextual signals, produce an explainable investigation and reflect a later verified status change.

### 1.3 What the demo does not prove

Real-time access to Nigerian banks; ability to determine credit from an NIP timeout; production availability, regulatory certification, general root-cause accuracy, reliable extraction of arbitrary documents, statistically

established superiority over incumbent vendors, measured reductions in Nigerian bank complaints, or automated dispute settlement. Simulated metrics shall be clearly labeled.

## 2. Cedar's fictional banking environment

### 2.1 Institutional identities and domain

- **Cedar Bank:** fictional sending bank and owner of the customer-facing app.
- **Elm Bank:** fictional receiving bank in the hero transfer.
- **Birch Bank:** optional second receiving bank to generate negative-control transactions.
- **Cedar Network Simulator:** NIP-inspired **mock** switch. It must use fictional institution identifiers, never claim to implement the exact NIBSS protocol or statuses.
- Accounts use clearly artificial display identifiers such as `CED-ACC-001` and `ELM-ACC-009`. Never use plausible live account numbers in fixture data.
- Currency is NGN; store money as **integer kobo** (₦50,000 = `5000000` kobo). All timestamps are ISO 8601 with offset and UTC storage.

### 2.2 Logical bank systems and their authority

| Logical system | What it does | Authoritative for | Emits / stores |
|---|---|---|---|
| Identity & Accounts | Demo users, accounts, role-scoped visibility | Cedar account ownership | Account/profile records, auth events |
| Core Banking | Account status and available balance projection | Cedar-maintained customer account state | Account snapshots |
| Ledger | Balanced financial journal postings | Cedar debit and reversal posting | LedgerJournal, LedgerEntry |
| Transfer Orchestrator | Validates and coordinates attempts | Cedar processing stage | Transfer, TransferAttempt, workflow events |
| Payment Gateway G2 | Makes outbound network requests | What Cedar sent / received | Message and timeout events |
| Mock Payment Network | Delivers, delays or drops synthetic messages | Simulated network event truth *within its restricted subsystem* | Request / response records |
| Elm/Birch Recipient Banks | Local synthetic beneficiary ledgers | Recipient credit, when disclosed through allowed messages | Recipient ledger and status responses |
| Reconciliation | Compares sender- and network-visible records | Discrepancy identification, not arbitrary final credit | Recon events / mismatch flags |
| Observability | Telemetry about services and routes | Observed metrics, not transfer finality | ServiceEvent, RouteMetric |
| Customer Support | Complaints, case status and history | Customer-reported experience | Complaint, ComplaintEvent |
| Bank Incident Desk | Bank-verified incident decisions | Reviewed incident statements | Incident review, case actions |
| Documents/Inventory | Cedar's policies, inventories, past incident reports | What a source says; not automatic external truth | KnowledgeSource, chunks, citations |

Cedar may be **one NestJS app** with separately organized modules. The source boundaries are logical and enforced by permissions, schemas and adapters. Do not claim physical microservice deployment.

### 2.3 Dependency map for the initial fixture

Cedar Customer App → Cedar Transfer Orchestrator → Cedar Gateway G2 → Mock Network → Elm Bank Receive Handler. Cedar's Ledger is invoked by the orchestrator. Observability watches orchestrator/gateway/network **observables**; it does not have automatic visibility into Elm's private ledger. Cedar Customer Support produces complaints; the bank incident desk reviews Veriq recommendations.

Add ROUTE-CED-ELM-G2 for Elm and ROUTE-CED-BIRCH-G3 for Birch. Monitoring should have normal samples for both routes to support a meaningful negative control.

## 3. Financial correctness and transfer simulation

### 3.1 Accounting model

Cedar maintains a **double-entry ledger**. Every posted journal consists of >= 2 entries, sum(debit_kobo) = sum(credit_kobo), and the journal is posted atomically. Ledger entries are append-only. The customer balance projection is derived from postings or atomically updated as part of the same database transaction. No floating-point amounts. No direct frontend or AI balance updates.

For Cedar's simplified transfer accounting, debiting the customer's deposit account liability is balanced against a dedicated OUTGOING_TRANSFER_CLEARING account. A verified reversal creates a *new compensating journal*. The recipient bank keeps a separate synthetic ledger. We model message exchange and customer credits, not full interbank settlement/net-position accounting.

### 3.2 Processing stage vs final outcome

**Processing stage** (historical events): CREATED, VALIDATED, DEBIT_POSTED, SUBMITTED, ACKNOWLEDGED, RESPONSE_TIMEOUT, STATUS_QUERIED, COMPLETED. Stages describe observed progression; timeout is an event, not proof of failure.
**Financial outcome** (separate knowledge): UNKNOWN, CREDIT_CONFIRMED, FAILURE_CONFIRMED, REVERSAL_POSTED.

Rules:

- CREDIT_CONFIRMED requires an eligible recipient-credit confirmation or authorized status evidence with defined semantics.
- FAILURE_CONFIRMED requires eligible definitive failure evidence, not the passage of time alone.
- REVERSAL_POSTED requires a confirmed Cedar compensating ledger journal.
- An internal scenario may know the true recipient outcome, but **Cedar's sender-visible status** and Veriq must remain UNKNOWN while the response is missing.
- Later evidence may advance knowledge; previous event facts remain immutable. Conflicting authoritative evidence opens a discrepancy for human review rather than being silently overwritten.

### 3.3 Transfer command sequence

1. Customer authenticated in Cedar; obtains demo balance and list of fictional banks.
2. Customer enters fictional beneficiary ID; the name-enquiry endpoint resolves it within the synthetic recipient-bank dataset.
3. Customer submits amount, beneficiary, narration and an Idempotency-Key (server-generated in client logic per intended transfer).

4. Orchestrator validates account ownership, active account, positive amount, sufficient balance and demo limits; creates transfer + attempt.
5. In one Cedar database transaction, reserve/post the relevant debit journal, mark transfer DEBIT_POSTED, append a durable event/outbox item.
6. Gateway submits synthetic payment instruction to the mock network; network processes a **scenario-selected** outcome.
7. Cedar records received response or timeout. It does not infer recipient credit from absence of response.
8. A permitted later status query may return new sender-visible outcome evidence. The backend updates only on eligible evidence.
9. Cedar complaint API can reference the same transfer ID and create a case even when FastAPI/AI is unavailable.

### 3.4 Required synthetic failure cases

| ID | Scenario | Hidden network / recipient truth | Cedar customer-visible result |
|---|---|---|---|
| S0 | Normal success | Elm credit posted, response delivered | CREDIT_CONFIRMED |
| S1 | Validation failure | Nothing sent or debited | Rejected command; no debit |
| S2 | Hero: credited, reply dropped | Elm credit **posted**, success reply lost | **Debited; UNKNOWN** until later authorized query |
| S3 | Definite failure + reversal | No Elm credit; failure eligible, compensating journal posted | REVERSAL_POSTED only after journal |
| S4 | Route-wide response degradation | Controlled cluster of G2 response timeouts; varied underlying outcomes | Suspected route incident; individual transfers can remain UNKNOWN |
| S5 | Similar wording, unrelated route | Birch transfer failure independent of G2 | Must not be incorrectly linked to Elm route incident |
| S6 | Conflicting / late events | Authoritative messages contradict or arrive out of order | Discrepancy flag; human review |

**Hero fixture S2 must be deterministic.** On recording day, the customer cannot randomly receive S0 or S3 instead. Scenario choice exists only through operator controls, not a parameter trusted from ordinary customer requests.

### 3.5 Event-envelope contract

Every accepted operational event records: event_id, bank_id, event_type, source_system, occurred_at, observed_at, optional transfer_id, service_id, route_id, correlation_id, payload, source_record_id and schema_version. event_id is unique in its source scope; re-ingestion is idempotent. Store raw immutable event and separately maintain projections. An earlier occurred_at event may arrive later (observed_at): replay must not produce impossible financial state transitions.

## 4. Bank onboarding and knowledge acquisition

### 4.1 Supported sources in v1

Cedar's onboarding can provide **any relevant permitted information** architecturally. v1 demo implements these specific adapters to keep correctness demonstrable:

| Package | Source format | Processing |
|---|---|---|
| services.json | Structured JSON | Deterministic parse to service candidates |
| dependencies.csv | Structured CSV | Deterministic parse to documented edges; validate both endpoints |
| routes.json | Structured JSON | Deterministic parse, link service and route IDs |
| historical_incidents.md | Unstructured Markdown | AI extracts incident entities, dates, symptoms, reported causes and source quotes |
| operations_sop.txt | Unstructured plain text | AI extracts procedures and bounded action constraints |
| prior_complaints.csv | Structured CSV | Complaint records and reported symptoms (not financial truth) |
| baseline_metrics.jsonl | Structured event lines | Historical route metrics and timestamps |

PDF parsing is a P1 input adapter if reliable; scanned PDFs, images and OCR are not mandatory. The complete product architecture can extend the source registry to formats and connectors not implemented in v1.

### 4.2 Ingestion pipeline

**Register** (source_id, type, original file, checksum, submitting actor, time, permissions) → **Parse** (file-specific adapter) → **AI extraction** (only when useful; strict schema) → **Stage** claims and candidate edges → **Normalize** bank-scoped IDs → **Validate and reconcile** → **Accept/flag/reject** → **Materialize** queryable context → **Audit**.

A document's sentence is evidence that the document contains a claim; not automatic proof the claim itself is true. A bank-supplied service inventory is a declared source. A supported exact service-reference link can be accepted by deterministic rule after validations; an AI-inferred root cause cannot.

### 4.3 Claim and relationship trust model

Assertion statuses: STAGED, ACCEPTED_AS_SOURCE_CLAIM, VERIFIED, REJECTED, SUPERSEDED. Relationships have separate relation_status (CANDIDATE, DOCUMENTED, OBSERVED, VERIFIED, REJECTED). These are intentionally **not** banking transfer outcome enums.

Every claim/edge MUST include bank_id, subject, predicate, object/value, source_id, source_locator (row/line/chunk), recorded_at, optional valid_from/valid_to, extraction_method, and status. AI claims also require model_run_id and an evidence citation. Deduplicate by canonical entity identity and source claim keys, not purely by similar wording.

Proposed identity key for bank-specific objects: (bank_id, entity_type, canonical_external_id). If only a name is available, stage an unresolved entity candidate; do not merge based on names alone. Resolve duplicate aliases with typed matching and human confirmation when ambiguous.

### 4.4 Initial institution-specific context

The context contains KnowledgeEntity, KnowledgeRelationship, Assertion, Source, OperationalEvent, CurrentState, Incident and EvidenceLink records. It must be possible to answer:

- Which documented services and routes does Cedar operate?
- Which sources establish or merely assert a relationship?
- Which routes depend on Gateway G2?
- What is happening to G2 now, and how does that differ from its historical baseline?
- Which comparable incidents were previously documented or verified?
- Which claims are unknown, disputed, superseded or only suggested by AI?

No arbitrary free-form prompt is allowed to mutate this state. All writes go through the NestJS validation/authorization layer.

## 5. Continuous context and incident detection

### 5.1 Continuous means event-driven, not permanently-running LLM calls

Cedar emits events and updates metrics independently of complaints. Veriq consumes authorized, filtered streams or event batches, updates projections, evaluates simple rules, and initiates investigations when warranted. LLM calls happen for ingestion or relevant investigative tasks, not on every telemetry event.

### 5.2 Deterministic detector for the demo

Evaluate route-specific timeout rate in a sliding **5-minute** window. Raise a SUSPECTED incident only when **(a)** at least 20 attempts are observed in the window, **(b)** timeout rate >= 20%, **(c)** a credible baseline from >= 100 historical attempts has timeout rate <= 5%, **and (d)** the present rate exceeds baseline by >= 15 percentage points. Require shared route/service signatures, and avoid a second duplicate incident for the same active signature. These are **synthetic demo thresholds**, not empirically validated production banking policy.

Seed: Elm/G2 historical baseline = 2 timeouts / 100 attempts; degrading window = 7 timeouts / 25 attempts. Birch/G3 stays near baseline. Route anomaly creates an incident even before a customer complaint. Its status remains SUSPECTED unless human/institutional evidence confirms it.

### 5.3 Case-to-incident correlation

Match on authorized transfer → exact route_id and processing time window → error/timeout event signature, then test against suspected incident cohort. Plain-language similarity may provide *candidate* retrieval, but may never override contradictory route, identity, time or evidence constraints. Intentionally include Birch's unrelated complaint with similar wording; it must not link to the G2 incident.

### 5.4 Investigation lifecycle

REQUESTED → evidence bundle assembled → AI_PENDING (optional) → FINDINGS_READY → investigator review → REVIEWED → optional RESOLVED (for a case only when its actual outcome is sufficiently evidenced). An AI outage produces EVIDENCE_READY_AI_UNAVAILABLE; the customer case remains accessible. Incident status and individual transfer outcome remain distinct.

## 6. Role-based user experiences

### 6.1 Customer: Cedar banking app

Required screens: account home, beneficiary/name enquiry, amount + confirm transfer, transaction details, **Report a problem**, case tracking. The customer sees only their own account and case. A debited transfer with no conclusive recipient evidence shows **"Your account was debited. We are still confirming the final outcome."** The system must not claim it failed or promise a reversal/ETA on timeout alone.

### 6.2 Investigator: Veriq console

Required screens: onboarding/source review, living-context explorer (graph/table + provenance), operational overview (events/route health/suspected incidents), investigation workspace (facts/unknowns/hypotheses/source links/review). Investigator may approve or reject proposed relationships and record findings. Do not display unrelated customers' personally identifying records.

### 6.3 Demo operator (not a bank customer)

Operator may reset/seeding only outside ordinary customer routes, start/pause the event stream, arm scenario S2, reveal authorized status evidence at the prescribed time, and optionally speed up simulation. Operator cannot insert an AI finding into the dashboard as though generated by the live pipeline. Every operator action should appear in demo audit logs.

## 7. Ownership and execution boundaries

| Owner | Required deliverables | Must not own |
|---|---|---|
| **Daniel — Backend / integration** | Cedar bank modules; ledger; transfer/network simulator; seed & isolated ground truth; source registration & parsing; canonical context schema; deterministic validation, route detector, complaint workflow, evidence assembly; NestJS APIs and persistence; AI orchestration; tests. | AI prompt authoring or frontend layouts. |
| **Habeeb — AI** | FastAPI extract-knowledge, analyze-case, optionally analyze-incident; Pydantic typed I/O, controlled prompts, citation checking, bounded hypotheses; fallback/error behavior; evaluation samples. | Direct authoritative ledger writes, user auth, ground-truth reads. |
| **Usaamah — Frontend** | Cedar customer experience, Veriq onboarding and context viewer, operations/incident dashboard, case/evidence review; clear loading/error states; realistic but synthetic demo design; wired API calls. | Final financial-state inference or frontend-only enforcement of data access. |

All three agree on a **single frozen OpenAPI / JSON contract** before implementing in parallel. The companion Veriq_Cedar_API_Contract_v1.0.yaml is the contract source; this document is the narrative specification.

## 8. API and AI contracts

### 8.1 Public Cedar APIs (/v1/cedar)

| Method / path | Auth | Behavior |
|---|---|---|
| GET /accounts/me | Customer | Own balance and account state |
| GET /banks | Customer | Fictional recipient bank list |
| POST /name-enquiries | Customer | Resolve synthetic beneficiary |
| POST /transfers | Customer | Idempotent amount transfer; server controls scenario |
| GET /transfers/{transferId} | Customer owner | Processing evidence and final outcome knowledge |
| POST /complaints | Customer owner | Persist case even if AI unavailable; link own transfer |
| GET /complaints/{caseId} | Customer owner | Customer-safe status, updates and verified facts |

### 8.2 Investigator/Veriq APIs (/v1/veriq)

| Method / path | Behavior |
|---|---|
| POST /banks/{bankId}/sources | Upload/register file; start parse/extract/stage job |
| GET /banks/{bankId}/sources | List provenance, status, accepted/flagged claims |
| GET /banks/{bankId}/context | Summary of entity classes, edges, sources, uncertainty |
| GET /banks/{bankId}/sources/{sourceId}/claims | Show staged/accepted/rejected extracted assertions with original citations |
| POST /banks/{bankId}/assertions/{assertionId}/review | Investigator accepts or rejects a proposed claim/relationship with reason |
| GET /banks/{bankId}/relationships | Filtered, permission-aware graph view |
| GET /banks/{bankId}/operations | Route metrics, events and suspected incidents |
| GET /banks/{bankId}/incidents | Incident list |
| GET /banks/{bankId}/incidents/{incidentId} | Cohort, evidence, alternative hypotheses, review |
| GET /banks/{bankId}/cases/{caseId} | Investigator case, evidence timeline, findings |
| POST /banks/{bankId}/cases/{caseId}/investigate | Build evidence bundle; call AI; persist validated advice |
| POST /banks/{bankId}/incidents/{incidentId}/review | Human verified/rejected decision with audit |
| POST /banks/{bankId}/cases/{caseId}/review | Investigator reviews findings and authorizes case-level action/update |

For assertion review, a document-backed statement can be accepted **as a documented claim** without asserting external causal truth. No AI proposal can be marked VERIFIED solely by pressing Accept; verified operational claims require qualifying source or investigator review against evidence. Every review records actor, reason and evidence IDs.

### 8.3 Operator endpoints (/v1/demo)

POST /reset, POST /seed, POST /stream/start, POST /scenarios/{scenarioId}/arm, POST /scenarios/{scenarioId}/run, POST /transfers/{transferId}/reveal-status. Demo-operator authorization is mandatory and forbidden in a real production configuration. reveal-status does not write Veriq truth directly; it causes Cedar's mocked **authorized status-query pathway** to emit a new eligible status response.

### 8.4 FastAPI internal endpoints (/internal/ai)

POST /extract-knowledge, POST /analyze-case; optional POST /analyze-incident may reuse the analyze-case contract. The **NestJS backend** determines permissions and selects context. FastAPI receives bounded evidence with stable IDs; it cannot fetch privileged simulator ground truth. All output is advisory JSON; backend performs schema validation, source-ID verification, enum checks, and verification-state enforcement.

### 8.5 Mandatory AI extraction output shape

```
{
 "source_id": "SRC-INC-001",
 "claims": [
  {
   "claim_id": "CLM-001",
   "subject": {"type": "SERVICE", "external_id": "G2"},
   "predicate": "REPORTED_CAUSE",
   "object": "connection-pool misconfiguration",
   "source_locator": "historical_incidents.md#incident-14",
   "quoted_evidence": "investigation identified a connection pool misconfiguration",
   "classification": "SOURCE_CLAIM",
   "needs_review": true
  }
 ],
 "unresolved_mentions": [],
 "warnings": []
}
```

The service must never return claims with missing evidence pointers as established facts. The backend validates subject/type identifiers and stages ambiguous claims rather than accepting them.

### 8.6 Mandatory investigation output shape

```
{
 "case_id": "CASE-001",
 "financial_outcome": "UNKNOWN",
 "observations": [
  {"text": "Sender debit posted", "evidence_ids": ["EV-DEBIT-1042"]},
  {"text": "Gateway response timed out", "evidence_ids": ["EV-TIMEOUT-1042"]}
 ],
 "hypotheses": [
  {
   "text": "Route G2 degradation may explain the missing response",
   "evidence_ids": ["EV-METRIC-G2-01", "EV-TIMEOUT-1042"],
```

```
    "alternatives": ["Recipient may have been credited despite response loss"]
   }
  ],
  "unknowns": ["Authoritative beneficiary credit outcome"],
  "recommended_checks": ["Authorized status re-query", "Inspect G2 route telemetry"],
  "customer_safe_update_draft": "We are checking the final status of your transfer."
}
```


financial_outcome is a **copy of the authoritative outcome passed by NestJS** and must be checked for exact equality on return. AI cannot promote UNKNOWN to confirmed. Model confidence is not financial evidence. All evidence_ids must exist in the evidence bundle and be authorized for the requesting investigator.

#### 8.7 API-wide rules

Responses include stable id, timestamps, status enums and schema_version. Errors follow { "error": { "code": "...", "message": "..." } }. The backend must return 401 for absent session, 403 for unauthorized role, 404 for out-of-scope resources, 409 for invalid state transitions, and 422 for schema validation errors. Use opaque IDs for public-facing lookup. API and client should share TypeScript definitions generated from OpenAPI when feasible.

## 9. Canonical context model and persistence

#### 9.1 Minimum logical entities

Bank, DemoActor, Customer, Account, LedgerAccount, LedgerJournal, LedgerEntry, Transfer, TransferAttempt, PaymentMessage, OperationalEvent, RouteMetric, KnowledgeSource, KnowledgeChunk, KnowledgeEntity, KnowledgeRelationship, Assertion, Complaint, ComplaintEvent, Incident, IncidentCaseLink, Investigation, Finding, EvidenceRecord, EvidenceLink, ReviewDecision, AuditEvent, ScenarioRun, SimulatorGroundTruth.

- All records used in bank-specific context require a bank_id scope.
- For every assertion: preserve reference to original source + extracted portion + status + observation/valid times.
- For every transfer: record separate processing_stage, financial_outcome, last_evidence_id, version and idempotency key.
- For every investigation: preserve evidence-bundle digest, AI model/provider/version if used, timestamps and human review.
- Ground truth is in a separate Cedar-only restricted persistence namespace. It is never accessible through a Veriq query or AI payload.

#### 9.2 Required relationship types

DEPENDS_ON, ROUTES_VIA, HANDLED_BY, EMITTED_BY, OBSERVED_ON, REFERS_TO_TRANSFER, AFFECTS_ROUTE, POSSIBLY_RELATED_TO_INCIDENT, CONFIRMED_RELATED_TO_INCIDENT, SUPPORTED_BY_EVIDENCE, MENTIONS_ENTITY. Add additional typed relationships only with validated semantics. In particular, **do not** use CAUSED_BY unless a bank-authorized investigator has recorded a verified causal conclusion.

### 9.3 Source and timeline invariant

Never rewrite raw source content or raw operational events in place. Store occurred_at (event-time), observed_at (ingestion-time), and valid_from/valid_to when applicable. Maintain current projections separately. A historic query must not leak future knowledge: when reconstructing what Veriq knew at 10:07, omit evidence observed at 10:20 even if that evidence describes an earlier event.

## 10. Seeded fixtures and testing controls

### 10.1 Minimum deterministic fixture

- 1 sending institution (Cedar), 2 fictional receivers (Elm, Birch).
- 20 synthetic Cedar customers, >= 1 demo customer and >= 1 Elm beneficiary; all fake identifiers.
- 8–12 logical service objects, 2 routes, 1 key gateway G2.
- 100 historic Elm-route attempts with 2 synthetic timeouts; >= 100 historical attempts for Birch, low timeout rate.
- One streaming Elm-route window of 25 attempts with 7 timeouts (after stream start), before the hero complaint.
- One prior historical G2 incident report in Markdown, one SOP in text, structured inventory/dependency/route files, and at least one misleading/unrelated complaint.
- The hero customer starts with at least ₦100,000 and sends ₦50,000 to Elm.
- Private scenario truth: Elm credit posted, response dropped; later status query discloses CREDIT_CONFIRMED.

Fixture integrity: every referenced service, bank, route, transfer and evidence ID must resolve consistently. Fixed clock/seed + deterministic scenario run order produce the same visible results after /v1/demo/reset.

### 10.2 Mandatory tests

| Test | Expected result |
|---|---|
| T01 | Re-import same source content twice |
| T02 | AI proposes unknown service alias |
| T03 | Source says G2 dependency absent from inventory |
| T04 | Normal route receives transactions |
| T05 | G2 7/25 timeout vs 2/100 baseline |
| T06 | Birch complaint uses same wording |
| T07 | Transfer request retried with same idempotency key |
| T08 | Hero timed-out response, private credit exists |
| T09 | Complaint submitted while AI unavailable |
| T10 | AI invents unsupported evidence ID |
| T11 | Later allowed status response confirms credit |
| T12 | Customer A requests customer B's transfer |
| T13 | Out-of-order event arrives |

| Test | Expected result |
|---|---|
| T14 | Demo reset and replay |
| T15 | All monetary journals |
| T16 | Source-/role-limited graph retrieval |

**Definition of done:** Full automated test suite for T01–T16 plus one manually rehearsed recording with zero mocked frontend-only transitions. If time requires triage, T01–T02 can be simplified but T05, T07–T12 and the end-to-end demo must not be sacrificed.

## 11. One submitted demo: two acts, one uninterrupted narrative

**Demo title:** *Cedar knows its bank. Veriq knows what to investigate.*
**Recording format:** One end-to-end recording/video; no second submitted clip. Target approximately 4–5 minutes internally, and edit to the organizer's verified time limit before submission. The event-specific maximum length is **not established in this engineering specification**.

| Beat | Approx time | Visible action | Hidden/technical proof |
|---|---|---|---|
| 1 | 0:00–0:20 | Introduce Cedar + Veriq and state synthetic demo | Clearly label simulated bank and data |
| 2 | 0:20–1:00 | In Veriq, upload services.json, dependencies.csv, historical_incidents.md, operations_sop.txt | Source registration and parse jobs genuinely execute |
| 3 | 1:00–1:35 | AI extracts claims; backend validates deterministic edges and stages uncertain candidates | Source citations, candidate vs verified distinctions visible |
| 4 | 1:35–2:00 | Show living Cedar context: services, G2 route, past incident, evidence and uncertainty | Stored graph/context query; no hard-coded picture |
| 5 | 2:00–2:20 | Start/replay Cedar events; G2 timeouts rise, suspected incident appears **before complaint** | 7/25 vs 2/100 detector; Birch stays normal |
| 6 | 2:20–3:00 | Switch to Cedar customer: send ₦50,000 to Elm; debit appears; transfer outcome is inconclusive | Real NestJS transfer and ledger + mock network response drop |
| 7 | 3:00–3:25 | Customer files complaint using transfer reference | Saved case triggers evidence assembly |
| 8 | 3:25–4:10 | Open Veriq investigator view: timeline, G2 relation, past incidents, supported hypothesis, unknown outcome | AI output references real evidence IDs and existing context |

| Beat | Approx time | Visible action | Hidden/technical proof |
|---|---|---|---|
| 9 | 4:10–4:35 | Operator releases permitted status response; investigator reviews; customer sees confirmed update | Prior UNKNOWN advances to CREDIT_CONFIRMED only after eligible evidence |
| 10 | 4:35–4:50 | Close: one bank-specific context, two distinct triggers, safer investigation | State future vision without claiming production results |

Sequence rule: Act II uses the **same persisted bank_id and context version** built during Act I. The operational alert is allowed to precede the complaint, which then triggers its own case investigation. One cohesive story demonstrates both proactivity and customer-led investigation.

### Demo fallback and risk controls

- Preload only synthetic fixture resources. If a nonessential file extraction is slow, demonstrate a pre-imported **real persisted** record with its source ID; do not fabricate completed model output.
- Maintain a deterministic offline fixture replay of known API responses **only if disclosed as recorded**, rather than masquerading as a live integrated backend.
- AI service unavailable: show graceful AI_UNAVAILABLE state and evidence-only workflow; don't assert AI findings were produced.
- Network simulation may compress elapsed time but must preserve event order and display that the clock is simulated.
- Verify screen capture, resolution, secret masking and submission duration before recording.

## 12. Delivery plan and team integration gates

| Gate | Backend (Daniel) | AI (Habeeb) | Frontend (Usaamah) |
|---|---|---|---|
| G0: contract freeze | Routes, DTOs, enums, event envelope, fixtures | Pydantic schemas + 2 sample inputs | Typed client and 3 route shells |
| G1: Cedar functions | Accounts, ledger, recipient simulator, S0/S2 | Working extract-knowledge mock-contract response | Cedar account → transfer → pending UI |
| G2: context constructed | Sources, parsing, staging, deterministic edges, context query | Real extraction with cited output | Upload + source review + context explorer |
| G3: investigation | Detector, complaints, evidence bundle, AI call and validation | Evidence-grounded analyze-case | Operations/incident and case view |
| G4: unified demo | status reveal, audit, full replay, tests | degradation/unknown safety and response tests | full route integration + recording rehearsal |

Daily sync must answer: Are requests compatible with the frozen API? Is one full scenario reproducible? Which missing integration blocks the one demo? Avoid expanding scope while any G0–G4 gate is broken.

### Scope exclusions (v1)

Real account numbers, real credentials, real money, production banking integrations, NIP protocol emulation, actual regulator certification, autonomous transfers or reversals, nationwide banking monitoring, cards/loans/fraud workflows, end-to-end settlement accounting, custom model training, data warehouse, Kubernetes/Kafka, arbitrary format OCR, generalized graph DB, probabilistic causal guarantees, realistic enterprise SSO, mobile-native app.

## 13. Security and trust baseline

- All data synthetic. Demo logins clearly labeled. No PII from real bank users.
- Customer roles can read only their synthetic accounts, transfers and customer-safe case fields. Investor/investigator role is bank-scoped; operator controls are separately guarded.
- Enforce bank/tenant scoping server-side on every Veriq context and investigation read.
- Source files and AI prompts are untrusted; strip executable content, avoid tool execution from document text, validate MIME/size, and protect against prompt injection.
- Enforce maximum files/payload sizes and ingestion quotas appropriate to demo. Store document checksum and parsing status.
- Never send Cedar simulator private truth, undisclosed recipient ledger, access secrets, or unrelated personal records to AI.
- Keep model name/version, evidence IDs and human decisions in an auditable history.
- No automatic financial action based on a suggested AI finding.

## 14. Acceptance and changes

This document is the **final v1.0 implementation baseline** for team handoff, not a claim of external bank or hackathon organizer approval. It supersedes earlier conversational suggestions where they conflict: **one demo, two acts, Cedar's structured/unstructured onboarding, living bank-specific context, and ₦50,000 Elm transfer** are mandatory. The original Zecathon brief remains an important supporting product-research source but was not a finalized Cedar/Veriq backend contract.

Any change affecting API shape, source authority, financial state semantics, scenario truth, role boundaries or demo steps requires a short shared ADR: *old rule, proposed rule, reason, affected owner(s), updated test*. Cosmetic changes need no ADR. The team should not silently diverge from this baseline.

### Signoff checklist

- ☐ Daniel confirms banking model, API and ledger invariants.
- ☐ Habeeb confirms AI input/output fields and bounded evidence guarantees.
- ☐ Usaamah confirms customer and investigator screen journeys.
- ☐ Team verifies the organizer's actual video duration and track requirements.
- ☐ Complete Cedar → Veriq onboarding → monitoring → customer transfer → complaint → investigation → authorized confirmation → customer update in **one** deterministic run.

## Appendix A. Key invariants in one page

1. Cedar ≠ Veriq. Their permissions and authority remain separate even if they share one NestJS process.
2. Simulator ground truth ≠ bank-visible evidence ≠ AI findings.
3. Debited + timeout ≠ failed; timeout ≠ recipient not credited.
4. Every debit or reversal uses balanced journal entries, never floats or mutation of past postings.
5. Every accepted edge or claim has source and time provenance.
6. AI extracts and recommends; NestJS authorizes and validates; humans review consequential conclusions.
7. Complaint filing cannot fail solely because AI is offline.
8. Live context updates occur independently of complaints.
9. One video, two connected acts, same Cedar instance and knowledge base.
10. All numbers, identities, events, outcomes and benchmarks shown are simulated.

## Appendix B. Glossary

**Cedar:** fictional Nigerian digital bank and simulated data authority.
**Elm:** fictional beneficiary institution.
**Veriq:** bank-specific contextual intelligence infrastructure.
**Source claim:** what a record/document says, irrespective of independent confirmation.
**Observed event:** what a specified system recorded; not automatically a final financial outcome.
**Verified finding:** conclusion established by eligible evidence and/or authorized investigator action.
**Ground truth:** private simulator knowledge used to evaluate what actually happened.
**Living context:** persisted entities, relationships, operational states, sources, time and verification statuses.
**Evidence bundle:** scoped, permitted source records assembled for a specific investigation.
**Proactive incident:** anomaly discovered through operational events before customer complaint; remains suspected until validated.
**Idempotency:** replaying a command/event does not create a second financial effect.

## Appendix C. Provenance and references

**Source-derived from team PRD:** Zecathon_Track2_Final_Product_Brief.pdf, v1.0 dated 10 Oct 2026, especially §§6–11 (customer/incident journey; evidence authority; AI boundaries; NestJS/FastAPI/Postgres choices; immutable processing versus outcome state; synthetic evaluation).
**New, explicitly designed for this handoff:** Cedar/Elm bank models, simulator truth and ledger simplification, living-context onboarding schemas, route thresholds, consolidated two-act demo, route inventory, role responsibilities, endpoints, test plan and fixture counts. These are **team engineering design decisions**, not established facts about any operating Nigerian bank.
**Real-world implementation disclaimer:** Real interbank processing, banking controls and NIBSS-specific messages are more complex. Cedar is an intentionally bounded NIP-inspired simulator, not a faithful protocol or certified financial system.